Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester V)

Artificial Intelligence Application Development and Jira

Roll No.: T013	Name: Saqlain Khan
Class: TYIT	Batch: 1
Date of Assignment: 18-07-2026	Date/Time of Submission: 18-07-2026

PRACTICAL 3

AIM :- Problem Solving by Searching (Informed Search)

A) Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the solution using Greedy Best-First Search with the Manhattan Distance heuristic.

CODE :-

```
import heapq

# Display puzzle state

def print_state(state):

    for i in range(0, 9, 3):

        print(state[i], state[i+1], state[i+2])

    print()

# Manhattan Distance Heuristic

def heuristic(state, goal):

    distance = 0

    for num in range(1, 9):

        current = state.index(num)

        target = goal.index(num)

        x1, y1 = current // 3, current % 3

        x2, y2 = target // 3, target % 3

        distance += abs(x1-x2) + abs(y1-y2)

    return distance

# Generate next states

def get_neighbors(state):

    neighbors = []

    blank = state.index(0)

    moves = [-3, 3, -1, 1]
```

```
    for move in moves:
        new = blank + move
        if new >= 0 and new < 9:
            if blank % 3 == 0 and move == -1:
                continue
            if blank % 3 == 2 and move == 1:
                continue
            temp = list(state)
            temp[blank], temp[new] = temp[new], temp[blank]
            neighbors.append(tuple(temp))
    return neighbors
# Greedy Best First Search
def greedy(initial, goal):
    queue = []
    heapq.heappush(
        queue,
        (heuristic(initial, goal), initial, [])
    )
    visited = set()
    while queue:
        h, state, path = heapq.heappop(queue)
        if state == goal:
            return path + [state]
        if state not in visited:
            visited.add(state)
            for next_state in get_neighbors(state):
                heapq.heappush(
                    queue,
                    (heuristic(next_state, goal),
                    next_state,
                    path + [state])
                )
    return None
# Initial and Goal State
```

```
initial = (1,2,3,
           4,0,6,
           7,5,8)


goal = (1,2,3,
        4,5,6,
        7,8,0)
print("Initial State:")
print_state(initial)
print("Goal State:")
print_state(initial)
solution = greedy(initial, goal)
print("Solution:")
for step, state in enumerate(solution):
    print("Step", step)
    print_state(state)
print("Goal reached successfully.")
print("Saqlain Khan S013")
```

OUTPUT :-

```
Initial State:
1 2 3
4 0 6
7 5 8

Goal State:
1 2 3
4 0 6
7 5 8

Solution:
Step 0
1 2 3
4 0 6
7 5 8

Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0

Goal reached successfully.
Saqlain Khan S013
```

B) Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest path using A* search with the Manhattan Distance heuristic.

CODE :-

```
import heapq

# Display puzzle state

def print_state(state):

    for i in range(0, 9, 3):

        print(state[i], state[i+1], state[i+2])

    print()

# Manhattan Distance Heuristic

def heuristic(state, goal):

    distance = 0

    for num in range(1, 9):

        current = state.index(num)

        target = goal.index(num)



        x1, y1 = current // 3, current % 3

        x2, y2 = target // 3, target % 3

        distance += abs(x1-x2) + abs(y1-y2)

    return distance

# Generate next states

def get_neighbors(state):

    neighbors = []

    blank = state.index(0)

    moves = [-3, 3, -1, 1]

    for move in moves:

        new = blank + move

        if new >= 0 and new < 9:

            if blank % 3 == 0 and move == -1:

                continue

            if blank % 3 == 2 and move == 1:

                continue

            temp = list(state)

            temp[blank], temp[new] = temp[new], temp[blank]
```

```
            neighbors.append(tuple(temp))

    return neighbors

# A* Search

def astar(initial, goal):

    queue = []

    # f(n) = g(n) + h(n)

    heapq.heappush(queue, (heuristic(initial, goal), 0, initial, []))

    visited = set()

    while queue:

        f, g, state, path = heapq.heappop(queue)

        if state == goal:

            return path + [state]

        if state not in visited:

            visited.add(state)

            for next_state in get_neighbors(state):

                new_g = g + 1

                new_f = new_g + heuristic(next_state, goal)


                heapq.heappush(

                    queue,

                    (new_f, new_g, next_state, path + [state])

                )

    return None

# Initial and Goal State

initial = (1,2,3,

           4,0,6,

           7,5,8)

goal = (1,2,3,

        4,5,6,

        7,8,0)

print("Initial State:")

print_state(initial)

print("Goal State:")

print_state(goal)
```

```
solution = astar(initial, goal)

print("Shortest Path using A* Search:")

step = 1

for state in solution[1:]:

    print("Step", step)

    print_state(state)

    step += 1

print("Goal reached successfully.")

print("Total number of moves:", len(solution)-1)

print("Saqlain Khan S013")
```

OUTPUT :-

```
Initial State:
1 2 3
4 0 6
7 5 8

Goal State:
1 2 3
4 5 6
7 8 0

Shortest Path using A* Search:
Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0

Goal reached successfully.
Total number of moves: 2
Saqlain Khan S013
```

C) Given a weighted graph representing cities and distances between them, write a Python program to find the shortest path from Arad to Bucharest using A* search with a heuristic function (straight-line distance to destination).

CODE :-

```
import heapq

# Graph: city and distance to connected cities

graph = {

    'Arad': [('Zerind',75), ('Sibiu',140), ('Timisoara',118)],

    'Zerind': [('Arad',75), ('Oradea',71)],
```

```
    'Oradea': [('Zerind',71), ('Sibiu',151)],
    'Sibiu': [('Arad',140), ('Oradea',151), ('Fagaras',99), ('Rimnicu Vilcea',80)],
    'Timisoara': [('Arad',118), ('Lugoj',111)],
    'Lugoj': [('Timisoara',111), ('Mehadia',70)],
    'Mehadia': [('Lugoj',70), ('Drobeta',75)],
    'Drobeta': [('Mehadia',75), ('Craiova',120)],
    'Craiova': [('Drobeta',120), ('Rimnicu Vilcea',146), ('Pitesti',138)],
    'Rimnicu Vilcea': [('Sibiu',80), ('Craiova',146), ('Pitesti',97)],
    'Fagaras': [('Sibiu',99), ('Bucharest',211)],
    'Pitesti': [('Rimnicu Vilcea',97), ('Craiova',138), ('Bucharest',101)],
    'Bucharest': []
}
# Heuristic: straight-line distance to Bucharest
heuristic = {
    'Arad': 366,
    'Zerind': 374,
    'Oradea': 380,
    'Sibiu': 253,
    'Timisoara': 329,
    'Lugoj': 244,
    'Mehadia': 241,
    'Drobeta': 242,
    'Craiova': 160,
    'Rimnicu Vilcea': 193,
    'Fagaras': 176,
    'Pitesti': 100,
    'Bucharest': 0
}
def astar(start, goal):
    queue = []
    # queue contains: f_cost, g_cost, city, path
    heapq.heappush(queue, (heuristic[start], 0, start, [start]))
    visited = set()
    while queue:
```

```
        f_cost, g_cost, city, path = heapq.heappop(queue)
        if city == goal:
            return g_cost, path
        if city not in visited:
            visited.add(city)
            for neighbor, distance in graph[city]:
                new_g = g_cost + distance
                new_f = new_g + heuristic[neighbor]

                heapq.heappush(queue, (new_f, new_g, neighbor, path + [neighbor]))
    return None, []
cost, path = astar('Arad', 'Bucharest')
print("Shortest path using A* Search:")
print("Path:", " -> ".join(path))
print("Total Cost:", cost)
print("Saqlain Khan S013")
```

OUTPUT :-

```
Shortest path using A* Search:
Path: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total Cost: 418
Saqlain Khan S013
```